

k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transferring the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

```
In [19]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [20]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [21]: # Run some setup code for this notebook.
from __future__ import print_function

import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
```

```
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```
In [22]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs682/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may ca
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

# Don't forget to run get_datasets.sh, or this will throw an error
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

Clear previously loaded data.

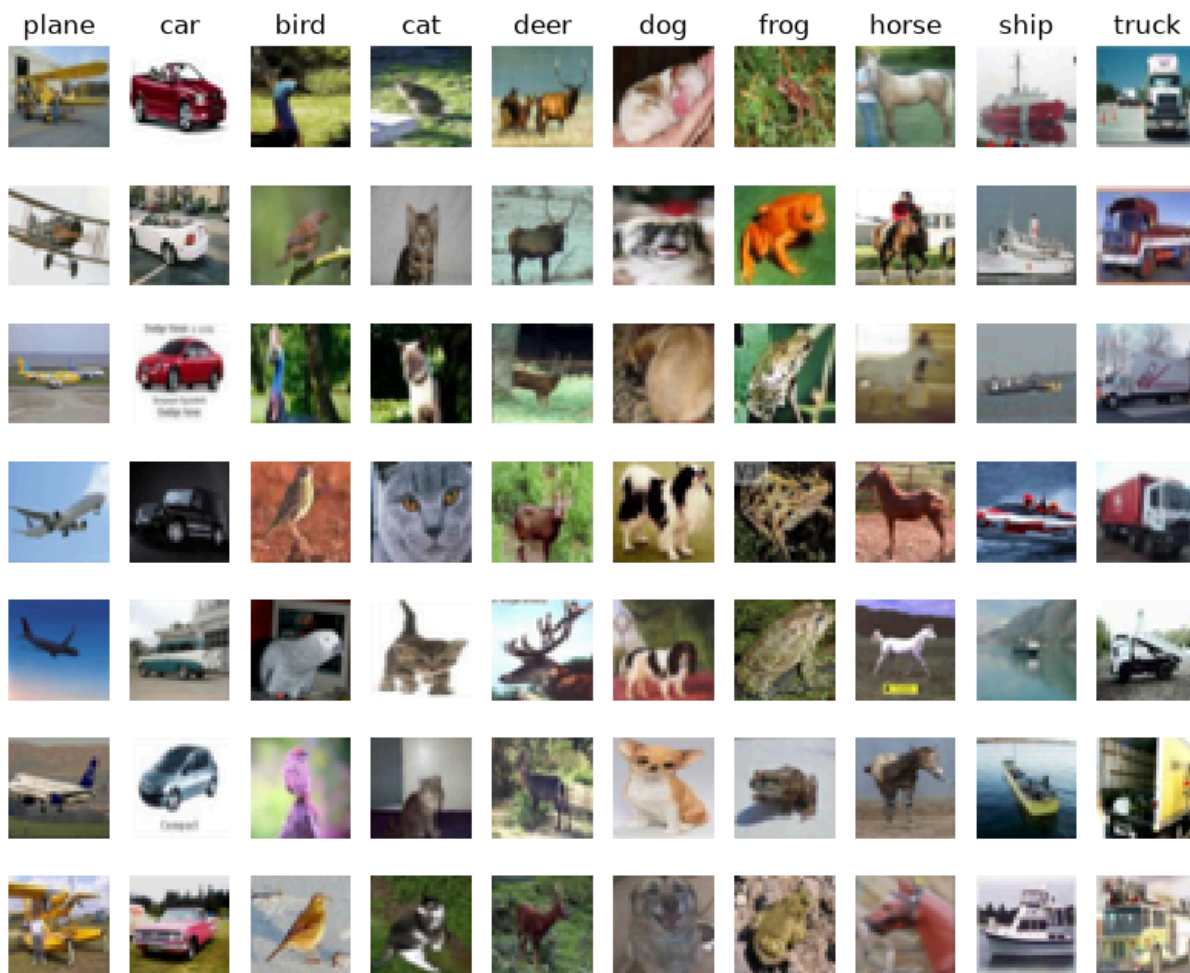
Training data shape: (50000, 32, 32, 3)

Training labels shape: (50000,)

Test data shape: (10000, 32, 32, 3)

Test labels shape: (10000,)

```
In [23]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 's
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
In [24]: # Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]
```

```
In [25]: # Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)

(5000, 3072) (500, 3072)
```

```
In [26]: from cs682.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are N_{tr} training examples and N_{te} test examples, this stage should result in a $N_{te} \times N_{tr}$ matrix where each element (i,j) is the distance between the i -th test and j -th train example.

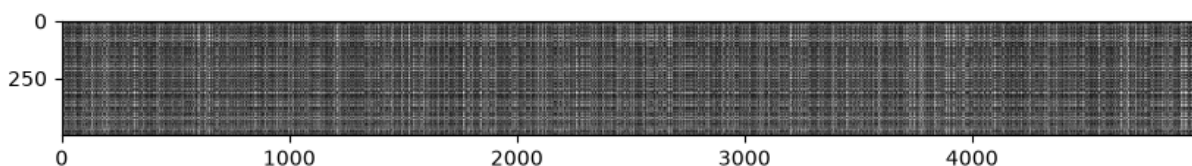
First, open `cs682/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

```
In [27]: # Open cs682/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

(500, 5000)

```
In [28]: # We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question #1: Notice the structured patterns in the distance matrix, where some rows or columns are visible brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Answer:

- **Bright rows:** a row is one *test* image. A bright row means that test image is far (in raw-pixel L2 distance) from almost every training image. This happens for test

images whose pixel statistics are unusual compared with the training set, e.g. very bright or very dark images, strong uniform backgrounds (white sky, black background), or unusual colors/high contrast. Because L2 on raw pixels is dominated by overall intensity and color, such images are far from everything.

- **Bright columns:** a column is one *training* image. A bright column means that training image is far from almost every test image, for the same reason: it is an atypical (very bright, very dark, or unusually colored) training example that no test image resembles at the pixel level.

```
In [29]: # Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 137 / 500 correct => accuracy: 0.274000

You should expect to see approximately 27% accuracy. Now let's try out a larger `k`, say `k = 5`:

```
In [30]: y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 139 / 500 correct => accuracy: 0.278000

You should expect to see a slightly better performance than with `k = 1`.

Inline Question #2: We can also use other distance metrics such as L1 distance. The performance of a Nearest Neighbor classifier that uses L1 distance will not change if (Select all that apply.):

1. The mean image of the training set is subtracted from the data.
2. All pixel values are divided by 255.
3. One mean is subtracted and the data is divided by one standard deviation, both computed over all pixel values.
4. Each color channel is standardized using its own mean and standard deviation (common in computer vision).
5. The coordinate axes for the data are rotated.
6. None of the above.

Answer: 1, 2, 3

Explanation: The L1 distance is $d(x, z) = \sum_p |x_p - z_p|$. A 1-NN (or k-NN) prediction only depends on the *ranking* of distances.

1. Subtracting the same mean image μ from every point:
 $|(x_p - \mu_p) - (z_p - \mu_p)| = |x_p - z_p|$, so all distances are unchanged.
2. Dividing every pixel by 255 scales every distance by the same factor $1/255$, so the ranking is unchanged.
3. Subtracting one scalar mean and dividing by one scalar std is a shift (cancels) followed by a uniform scale by $1/\sigma$, so the ranking is unchanged.
4. **Changes** performance: each channel is divided by a *different* σ_c , so the three channels are weighted differently in the sum and the ranking of neighbours can change.
5. **Changes** performance: L1 is not rotation invariant (unlike L2). E.g. $x = (1, 0)$ has L1 norm 1, but rotated by 45° it becomes $(1/\sqrt{2}, 1/\sqrt{2})$ with L1 norm $\sqrt{2}$, so relative distances and neighbours can change.

```
In [31]: # Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words, reduce
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('Difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

Difference was: 0.000000

Good! The distance matrices are the same

```
In [37]: # Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('Difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

Difference was: 0.000000

Good! The distance matrices are the same

```
In [33]: # Let's compare how fast the implementations are
def time_function(f, *args):
    """
```

```

    Call a function f with args and return the time (in seconds) that it took
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# you should see significantly faster performance with the fully vectorized

```

Two loop version took 13.567706 seconds

One loop version took 9.892429 seconds

No loop version took 0.327388 seconds

Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value $k = 5$ arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

```

In [34]: num_folds = 5
         k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

         X_train_folds = []
         y_train_folds = []
         #####
         # TODO:
         # Split up the training data into folds. After splitting, X_train_folds and
         # y_train_folds should each be lists of length num_folds, where
         # y_train_folds[i] is the label vector for the points in X_train_folds[i].
         # Hint: Look up the numpy array_split function.
         #####
         X_train_folds = np.array_split(X_train, num_folds)
         y_train_folds = np.array_split(y_train, num_folds)
         #####
         #
         #                                     END OF YOUR CODE
         #####

         # A dictionary holding the accuracies for different values of k that we find
         # when running cross-validation. After running cross-validation,
         # k_to_accuracies[k] should be a list of length num_folds giving the different
         # accuracy values that we found when using that value of k.
         k_to_accuracies = {}

         #####

```

```

# TODO:
# Perform k-fold cross validation to find the best value of k. For each
# possible value of k, run the k-nearest-neighbor algorithm num_folds times,
# where in each case you use all but one of the folds as training data and t
# last fold as a validation set. Store the accuracies for all fold and all
# values of k in the k_to_accuracies dictionary.
#####
for k in k_choices:
    k_to_accuracies[k] = []

for fold in range(num_folds):
    # fold `fold` is the validation set, the rest is the training set
    X_val_cv = X_train_folds[fold]
    y_val_cv = y_train_folds[fold]
    X_tr_cv = np.concatenate([X_train_folds[j] for j in range(num_folds) if
    y_tr_cv = np.concatenate([y_train_folds[j] for j in range(num_folds) if

    cv_classifier = KNearestNeighbor()
    cv_classifier.train(X_tr_cv, y_tr_cv)
    # distances do not depend on k, so compute them once per fold
    cv_dists = cv_classifier.compute_distances_no_loops(X_val_cv)

    for k in k_choices:
        y_val_pred = cv_classifier.predict_labels(cv_dists, k=k)
        accuracy = np.mean(y_val_pred == y_val_cv)
        k_to_accuracies[k].append(accuracy)
#####
#                                     END OF YOUR CODE
#####

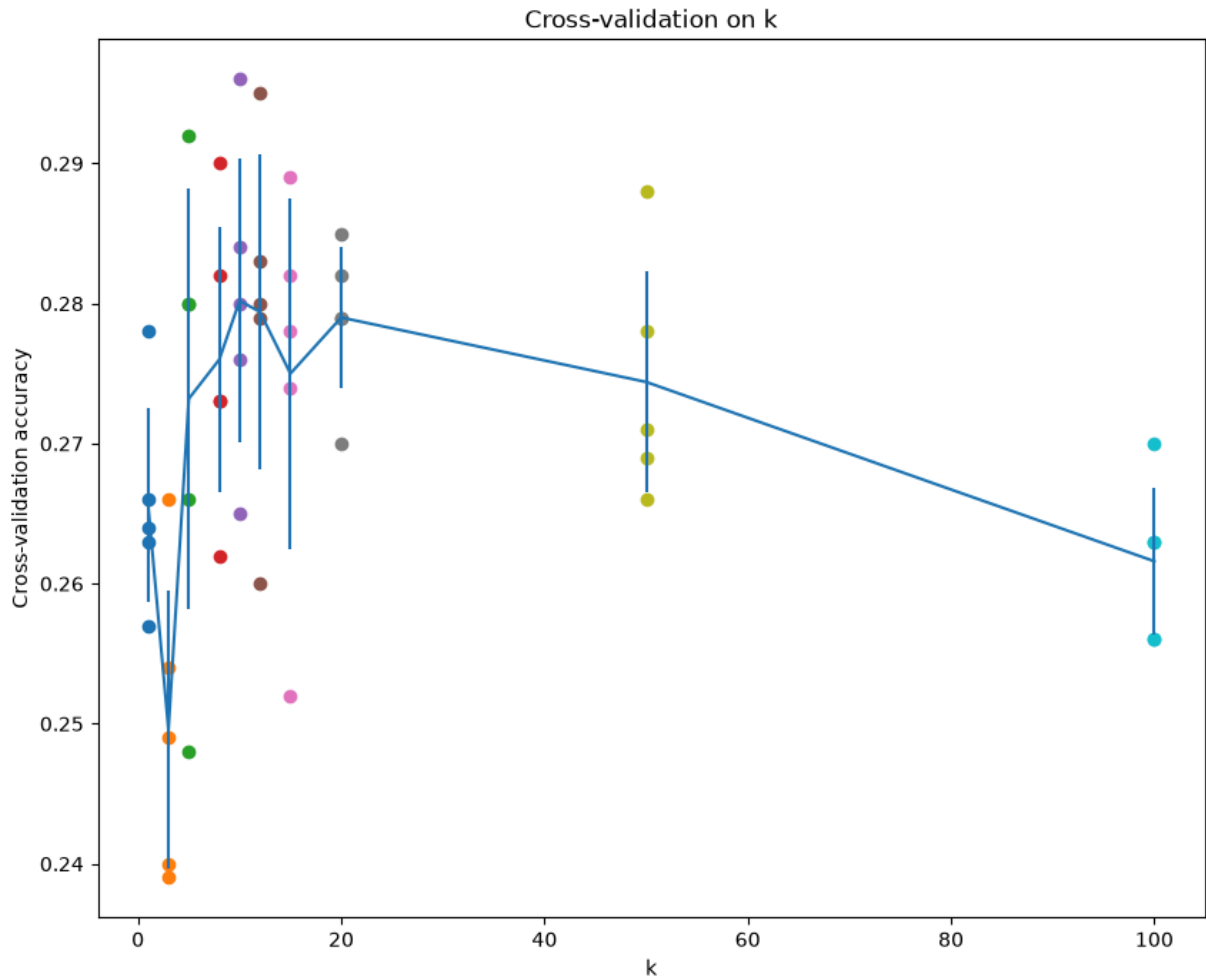
# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))

```

```
k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.263000
```

```
In [35]: # plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)
```

```
# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.items)])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.items)])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()
```



```
In [36]: # Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = 10
```

```
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)
```

```
# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 141 / 500 correct => accuracy: 0.282000

Inline Question 3 Which of the following statements about k -Nearest Neighbor (k -NN) are true in a classification setting, and for all k ? Select all that apply.

1. The training error of a 1-NN will always be better than or equal to that of 5-NN.
2. The test error of a 1-NN will always be better than that of a 5-NN.
3. The decision boundary of the k -NN classifier is linear.
4. The time needed to classify a test example with the k -NN classifier grows with the size of the training set.
5. None of the above.

Answer: 1, 4

Explanation:

1. **True.** With 1-NN every training point is its own nearest neighbour (distance 0), so the training error is 0 (assuming no duplicate images with different labels). 5-NN can only have training error ≥ 0 , so 1-NN is always better than or equal.
2. **False.** 1-NN overfits to noise in the training set; a larger k often generalises better (in this notebook $k = 5$ and the cross-validated k beat $k = 1$ on test data). It is not "always" either way.
3. **False.** The k -NN decision boundary is formed from pieces of the perpendicular bisectors between training points (a Voronoi-style partition); it is piecewise linear and in general highly non-linear, not a single linear boundary.
4. **True.** k -NN has no real training step; to classify one test point it must compute the distance to every training point, so the cost grows linearly with the training set size, $O(N \cdot D)$.